

NoSQL: АРХИТЕКТУРА ПЛАТФОРМЫ ОНЛАЙН-ОБУЧЕНИЯ

СРО

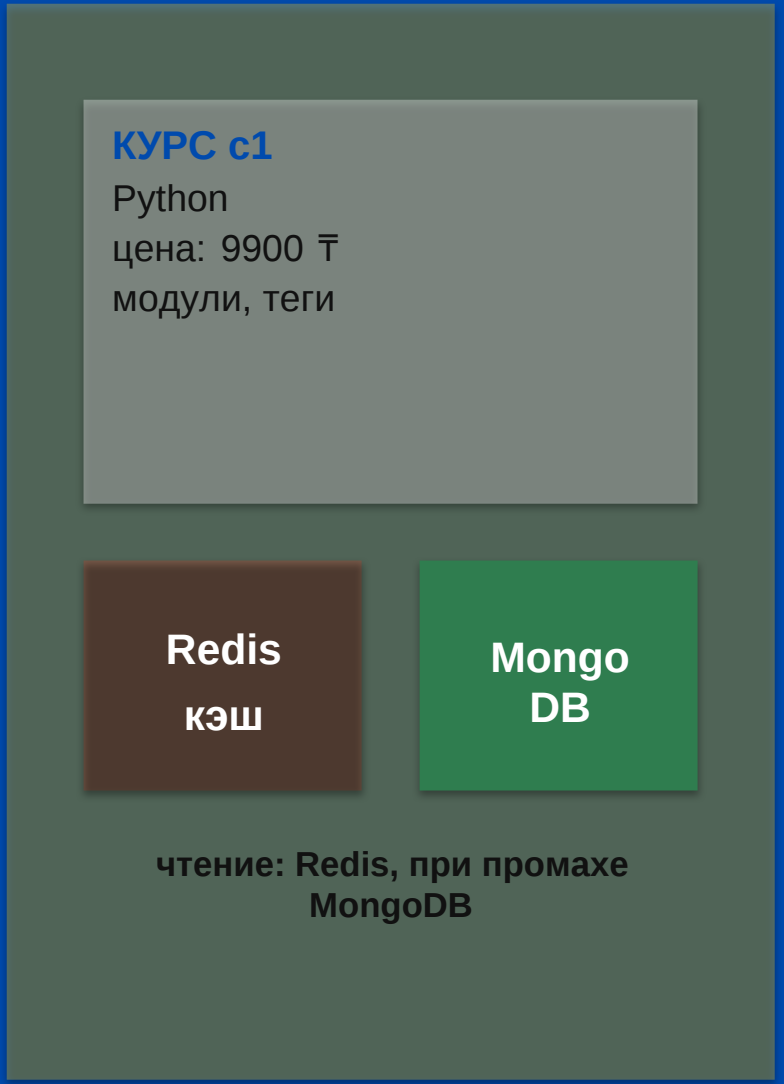
MongoDB+Redis·ВАРИАНТ 20

Жолдас Қайсар ИС 24-22

ЗАЧЕМ НУЖЕН ПРОЕКТ

Зачем нужно:

- **Скорость: карточки курсов читаются намного чаще, чем меняются; кэш в Redis снимает нагрузку с основной базы.**
- **Актуальность: после смены цены ключ в кэше удаляется, пользователь не видит устаревшее значение.**
- **История: в записи на курс хранится snapshot: название и цена на момент записи.**
- **Цель проекта: разработать прототип подсистемы данных платформы онлайн-обучения на MongoDB + Redis и обосновать выбор NoSQL.**



ПРО ЧТО ПРОЕКТ

1. **Хранение данных:** пользователи, курсы с вложенными модулями и записи на курсы лежат в MongoDB.
2. **Кэш cache-aside:** карточка курса читается из Redis, при промахе из MongoDB, TTL 300 с.
3. **Инвалидирование:** при смене цены ключ в Redis удаляется, следующее чтение берёт свежие данные.
4. **Рейтинг и аналитика:** топ курсов в Redis Sorted Set, 8 запросов к MongoDB и 4 теста.

MongoDB + Redis



ТРЕБОВАНИЯ И ACCESS PATTERNS

Функциональные (8)

- CRUD курсов
- Карточка курса с кэшем
- Запись на курс без дублей
- Snapshot названия и цены
- Удаление записи и курса
- Фильтры: категория, цена, теги
- Курсы студента с прогрессом
- Топ курсов и аналитика

Нефункциональные (4)

- Чтение из кэша быстрее БД (замер)
- Нет устаревшей цены после изменения
- Запуск по README, без секретов
- Индексы и уникальность записи

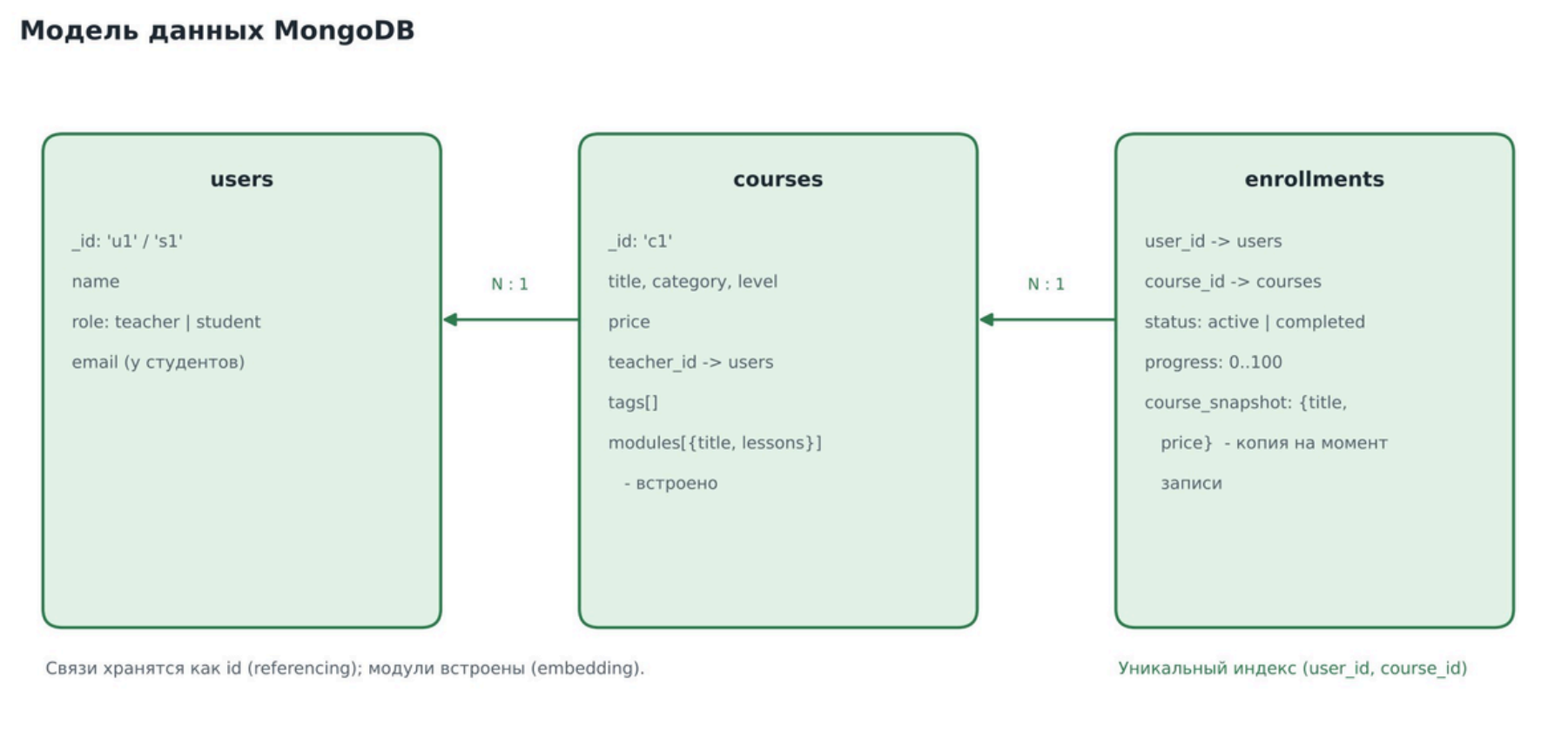
Access Patterns (8)

- Высокий приоритет: карточка курса, запись, смена цены, курсы студента
- Средний: фильтры, топ курсов (Redis)
- Низкий: аналитика (aggregation)

ВЫБОР ТЕХНОЛОГИИ И ADR

- **MongoDB:** карточкакурса - одиндокумент с модулями и тегами,гибкаясхема,aggregation pipeline.
- **Redis:** быстрые чтения, TTL, Sorted Set для рейтинга. Не источник истины.
- **PostgreSQL:** тоже подошёл бы; не выбран ради гибкой карточки (честное замечание в ADR-1).
- **Cassandra:** нагрузка не write-heavy, запросы произвольные - не подходит.
- **Neo4j:** обходов графа в паттернах доступа нет, отложен (ADR-5); вернуться, если появятся рек омендации.
- **ADR-2:** инвалидирование + TTL лучше, чем только TTL: нет окна устаревшей цены до 5 минут.
- **ADR-3:** snapshot цены в записи: цена курса меняется, история остаётся прежней.

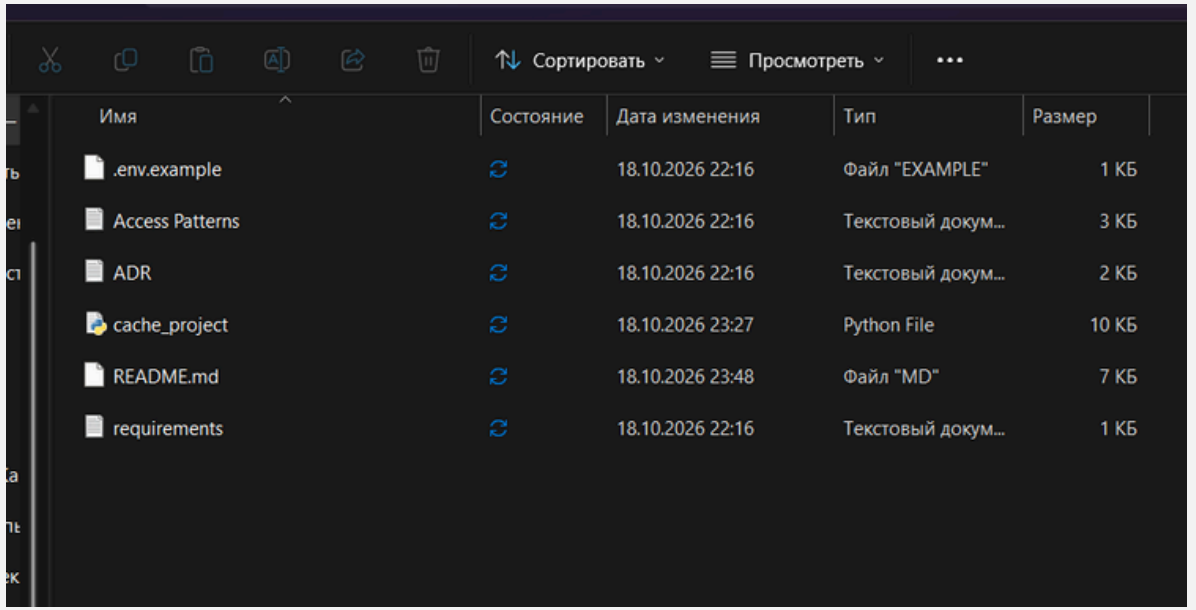
МОДЕЛЬ ДАННЫХ



- Модули курса встроены (embedding); записи вынесены в отдельную коллекцию со ссылками user_id и course_id (referencing).
- Уникальный индекс (user_id, course_id) запрещает повторную запись; course_snapshot хранит цену на момент записи.

АРХИТЕКТУРА И СТРУКТУРА ПРОЕКТА

- **cache_project.py -** единственный скрипт: seed, get_course, update_price,enroll,delete, queries, tests, timing.
- **README.md -** запуск и варианты установки MongoDB и Redis.
- **requirements.txt, .env.example -** pymongo, redis; адреса баз без секретов.
- **Access Patterns.txt, ADR.txt -** паттерны доступа и решения.
- **docs/ -** схемы, отчёт, презентация.



СРЕДА РАЗРАБОТКИ И РЕАЛИЗАЦИЯ

- **Python 3 и PyCharm:** одинскрипт,библиотеки pymongo и redis-py.
- **MongoDB Community Server:** локально, как служба Windows.
- **Redis (Memurai):** локально, порт 6379.
- **Cache-aside:** GET, при промахе find_one и SETEX на 300 с.
- **Инвалидирование:** update_one, затем DEL ключа.
- **Данные:** 39 синтетических записей, создаются функцией seed().

```
cached = r.get(key)
if cached:
      return json.loads(cached), "cache"
doc = db.courses.find_one(
      {"_id": course_id})
if doc:
      r.setex(key, TTL, json.dumps(doc))
return doc, "mongo"
```

Фрагмент get_course (cache_project.py)

РЕЗУЛЬТАТЫ ЗАПРОСОВ И ТЕСТОВ

Q5. Топ-3 курсапозаписям($group+$lookup)

Курс	Записей	Ср. прогресс, %
FastAPI	6	87
MongoDB	5	43.4
ML	5	95

Q7. Нагрузка преподавателей (двойной $group)

Преподаватель	Ст	Записей
Айдар	удентов: 8	13
Мадина	7	11

- 8 запросов: find, $group, $lookup, $cond, ZREVRANGE
- 4 теста: кэш и TTL, инвалидирование, snapshot и дубль, удаление курса

```
1) Первое чтение: mongo | повторное: cache
2) После смены цены: 12345 | источник: mongo (кэш сброшен)
3) Повторная запись на курс отклонена: True
4) Топ популярных курсов из Redis: [('c4', 6), ('c5', 5), ('c3', 5)]

Q1. Курсы категории Databases по цене:
   {'_id': 'c3', 'title': 'MongoDB', 'price': 14900}
   {'_id': 'c2', 'title': 'NoSQL', 'price': 19900}

Q2. Средняя цена по категориям ($group):
   {'_id': 'Data Science', 'avg_price': 29900.0}
   {'_id': 'Databases', 'avg_price': 17400.0}
   {'_id': 'Web', 'avg_price': 14900.0}
   {'_id': 'Programming', 'avg_price': 12345.0}

Q3. Топ-3 курса по записям ($group + $lookup):
   {'avg_progress': 66.0, 'title': 'FastAPI', 'enrollments': 6}
   {'avg_progress': 42.8, 'title': 'MongoDB', 'enrollments': 5}
   {'avg_progress': 69.0, 'title': 'ML', 'enrollments': 5}
```

```
Q4. Завершили курс по категориям ($cond):
   {'_id': 'Web', 'total': 6, 'completed': 0}
   {'_id': 'Databases', 'total': 9, 'completed': 0}
   {'_id': 'Data Science', 'total': 5, 'completed': 0}
   {'_id': 'Programming', 'total': 4, 'completed': 0}

1000 чтений карточки: MongoDB 419.6 мс, через Redis 117.4 мс

Process finished with exit code 0
```

ЗАКЛЮЧЕНИЕ

- **Итоги проекта:**
  - Разработан работающий прототип подсистемыданных наMongoDB+Redis; выбор обоснован в ADR-1…ADR-5.
  - Документная модель со snapshot цены и cache-aside с инвалидированием решают задачу: быстрая выдача без устаревшей цены.
  - Выполнены требования варианта 20: схема данных, 8 запросов, 4 тестовых сценария, README, отчёт и презентация.
- **Ограничения:**
  - Один скриптбезRESTAPI;принедоступномRedisскриптзавершается;неттранзакциймежду коллекциями; данные синтетические.
- **Практическая ценность:** основа для каталога курсов платформы онлайн-обучения.
- **Спасибо за внимание!**

Ссылка на репозиторий: https://github.com/Kays000/NoSQL-SRO